



نظري

6

5

600 sp

كلية الهندسة المعلوماتية

السنة الأولى

المصفوفات

د.مدحت الصوص-د.هبة حاتم



RB Informatics; 18/01/2025

برمجة C++

مع نهاية المحاضرة السابقة نكون قد أنهينا الأساسيات في لغة C++ وما سنأخذه في المستقبل هو مجرد أمور تساعد في كتابة البرامج بشكل أسهل وسنبدأ اليوم بالمصفوفات :
إذا أردنا حساب المتوسط الحسابي ل 100 عدد باستخدام برنامج فمن غير المنطقي تعريف 100 متحول ،
مثال :

```
int x1,x2,x3,.....x100;
cin>>x1>>x2>>x3>>.....>>x100;
int z=x1+x2+x3+.....+x100;
cout<<z/100.0;
```

- طبعاً الحل السابق سيء.

هل ممكن استعمال For؟ لنجرب:

```
for(int i=1;i<=100;i++)
    cin>>xi;
```

- طبعاً الحل مرفوض لا يمكن كتابة xi.

- أتت المصفوفات لإيجاد حل للمشكلة السابقة.

المصفوفات الأحادية

■ نعرف المصفوفات برمجياً بالشكل:

[عدد عناصر المصفوفة]

نوع عناصر المصفوفة

اسم المصفوفة

int

x

[100]

فتكون بالشكل الآتي في البرنامج:

```
int x[100];
```

وبذلك نكون عرفنا مصفوفة اسمها x تحوي 100 عنصر من نوع البيانات int.

شرح مسط :

قمنا بحجز رف اسمه x، وعليه 100 علبة، كل علبة مستقلة عن غيرها بمحتواها ولكن تشترك جميع العلب بنفس نوع البيانات.

توضيح لما سبق:



رف المتحول x ويحوي 100 علبة بدءاً من 0



كيف يمكن الوصول إلى علبة محددة؟

يمكن الوصول لأي علبة عن طريق تسمية الرف مع رقم العلبة.

x[2]

مثال:

هي العلبة الثالثة في الرف x.

بالنسبة لي هي متحول يمكن أن أفعل به ما أشاء.

أمثلة:

هي نفسها x[4]=10

x[1+3]=10;

cin>> x[3];

x[3]=5+y;

x[1+x[3]];

x[3]=5+x[2];

بعد أن عرفنا كيفية تعريف المصفوفة لنعود لحل المشكلة السابقة برمجياً:

int x[100];

cin>>x[0]>>x[1]>>x[2]>>.....>>x[99];

int z=x[0]>>x[1]>>x[2]>>.....>>x[99];

cout<< z/100.

طبعاً الأسطر السابقة غير مجدية وهي غير مفيدة لأننا لم نستفد من المصفوفة بشكل مناسب.

ولنحل هذه المشكلة نستعمل حلقة تكرار لأنه لدينا 100 عملية إدخال، فنكتب:

int x[100];

for(int i=0;i<100;i++)

cin>>x[i];

int z=0;

for(int i=0;i<100;i++)

z=z+x[i];

cout<<z/100.0;

هذه الحلقة لجمع عناصر

المصفوفة بعد إدخالها

✍ حجز المصفوفة يكون بشكل متتالي (يخزن العنوان فقط ولا يخزن النهاية)

1. ماذا يحدث إذا قمنا بتخزين شيء خارج المصفوفة ؟ (مثل طول مصفوفة 100 و خزنا $a[100]$)

99% من اللغات سيعطي error، ولن يقوم ببناء البرنامج، ولسوء الحظ، في هذه اللغة يقوم ببناء البرنامج دون أن يعطي أي error.

2. كيف يصل الى أي علبه في أي مصفوفة؟

يقوم بتخزين عنوان المصفوفة ويحسب مكانها.

مثال:

لدينا المصفوفة `int x[100];`

إذا أردنا الوصول ل `x[3]` سوف يحسب بناء على العملية `3*4 byte` ثم يعطينا القيمة الموجودة (ينتقل `4 byte` تلو الآخر)

✍ لذلك إذا قمنا بوضع قيمة في مكان بعد آخر خانة سيحدث واحد من ثلاثة:

1. يكون المكان محجوز لبرنامج آخر ويعطي runtime error وهذه الحالة هي الأقل سوءاً لأنه من الممكن ان تكتشف المشكلة وتحلها.
2. يكون المكان فارغ ويدعنا ندخل بشكل طبيعي ويخزن نفس القيمة، ولكن المشكلة هنا أن الذاكرة غير محجوزة فمن الممكن أن تذهب القيمة، وهي حالة متوسطة السوء لأنك قد تخسر بعض البيانات فقط.
3. يكون المكان محجوز لمتحول معرف مسبقاً في البرنامج نفسه وبالتالي سيكون الإدخال فوق قيمة المتحول وهي الأكثر سوءاً لأنك تخسر البيانات بالإضافة إلى أن التخزين سيصبح على مكان آخر في برنامجك ولا تعلم مما يؤثر على عمل كامل البرنامج.

وكما قلنا سابقاً أن هذا الخطأ لا يحدث في أغلب اللغات الأخرى، لأنه يتم تخزين طول المصفوفة مع حجمها (أما هنا فقد خُزنت بداية الـ rاف فقط).

✍ من مزايا المصفوفات أنها ثابتة الطول والحجم، ولا يمكن أن تخزن مصفوفة فوق أخرى.

-- بما أنها ثابتة الطول والحجم فلا يمكنني أن أتيح للمستخدم إدخال حجم المصفوفة (عدد العلب فيها)، يجب أن يكون طول وحجم المصفوفة مُحددين أثناء تعريفها (أثناء البناء يجب أن تكون محددة الطول والحجم).

✍ من الممكن تعريف المصفوفة في البرنامج بالطرق التالية أيضاً:

1. `const int n=10;`

`int x[n]`

ويلزم وضع const قبل المتحول حتى تكون قيمته ثابتة طوال البرنامج.

2. `int x[]={5,6,10};`

سيعرف ان طول المصفوفة السابقة هو 3.

3. بعض المترجمات تقبل تحديد طول المصفوفة أثناء تنفيذ البرنامج والبعض الآخر لا يقبل مثال:

```
int n;
cin >> n;
int a[n];
```

❗ إذا أردنا تشكيل 6 مصفوفات وكل مصفوفة تحوي 1000 خانة نقوم ب :

```
int marks1[1000];
int marks2[1000];
.
.
.
int marks6[1000];
```

❗ وهنا أتت فكرة المصفوفة الثنائية

المصفوفات الثنائية

▪ نعرف المصفوفة الثنائية برمجياً:

نوع عناصر المصفوفة	اسم المصفوفة	[عدد الأسطر]	[عدد الأعمدة]
int	marks	[i]	[j]

فيكون حل المشكلة السابقة :

```
int marks[6][1000];
```

الأعمدة الأسطر

يمكننا تخيل شكلها على أنها خزانة وبها رفوف كل رف عليه عدد علب معين وطبعا كل الرفوف تحوي نفس عدد العلب.

	0	1	2	3	...			...	999
0									
1									
2									
3									
4									
5									
6									

❖ لاستدعاء علبة (خانة) معينة يكون بالشكل:

```
int marks[2][0];
```

وهي العلبة الأولى في الرف الثالث

❖ للإدخال نستعمل حلقتين واحدة للأسطر وواحدة للأعمدة (واحدة للعلب وواحدة للرفوف):

```
for (int j=0; j<1000; j++)
    for(int i=0; i<6; i++)
        cin>>marks[j][i];
```

ملاحظة: لا داعي لاستخدام الأقواس {} للحلقة الأولى لأنها تحوي تعليمة واحدة فقط.

- الإدخال السابق كان لكل سطر ثم الانتقال لسطر آخر وإدخال خاناته.
- إذا أردنا إدخال كل عمود ثم الانتقال إلى عمود آخر لإدخال خاناته :

```
for (int j=0; j<1000; j++)
    for(int i=0; i<6; i++)
        cin>>marks[i][j];
```

Good luck!

